Image Classification

Presented by Ranga Rodrigo

Slides are from the sources particularly from Justin Johnson, Derek Hoiem and Svetlana Lazebnik.

History of Deep Convolution Networks

- 1950s: neural nets (perceptron) invented by Rosenblatt
- 1980s - 1990s: Neural nets are popularized and then abandoned as being interesting idea but impossible to optimize or “unprincipled”
- 1990s: LeCun achieves state-of-art performance on character recognition with convolutional network (main ideas of today’s networks)
- 2000s: Hinton, Bottou, Bengio, LeCun, Ng, and others keep trying stuff with deep networks but without much traction/acclaim in vision
- 2010-2011: Substantial progress in some areas, but vision community still unconvinced
- Some neural net researchers get ANGRY at being ignored/rejected
- 2012: shock at ECCV 2012 with ImageNet challenge
- ...

Slide: Derek Hoiem

Image Classification

Input: image

Output: Assign image to one of a fixed categories

cat
dog
deer
bird
car
truck

```
[[127, 143, 142, 165, 170, 125, 119],
 [124, 129, 113, 114,  91,  84, 106],
 [104,  98,  37,  11,  65,  27,  70],
 [ 81,  83,  35,  12,  20,  59,  43],
 [ 74, 119, 114,  66,  60,  61,  64],
 [ 96, 108, 130, 136, 120, 114, 111]]
```

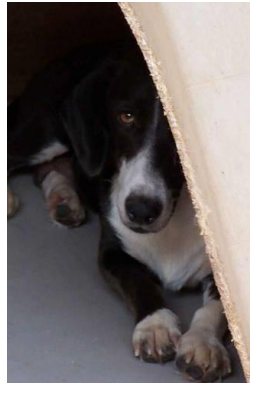
What the computer sees (semantic gap) ← what human and computer see

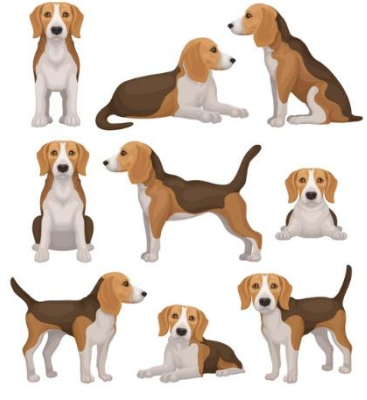
An image is just a big grid of numbers between [0, 255]:

E.g., $800 \times 600 \times 3$ (3 channels RGB)

Challenges

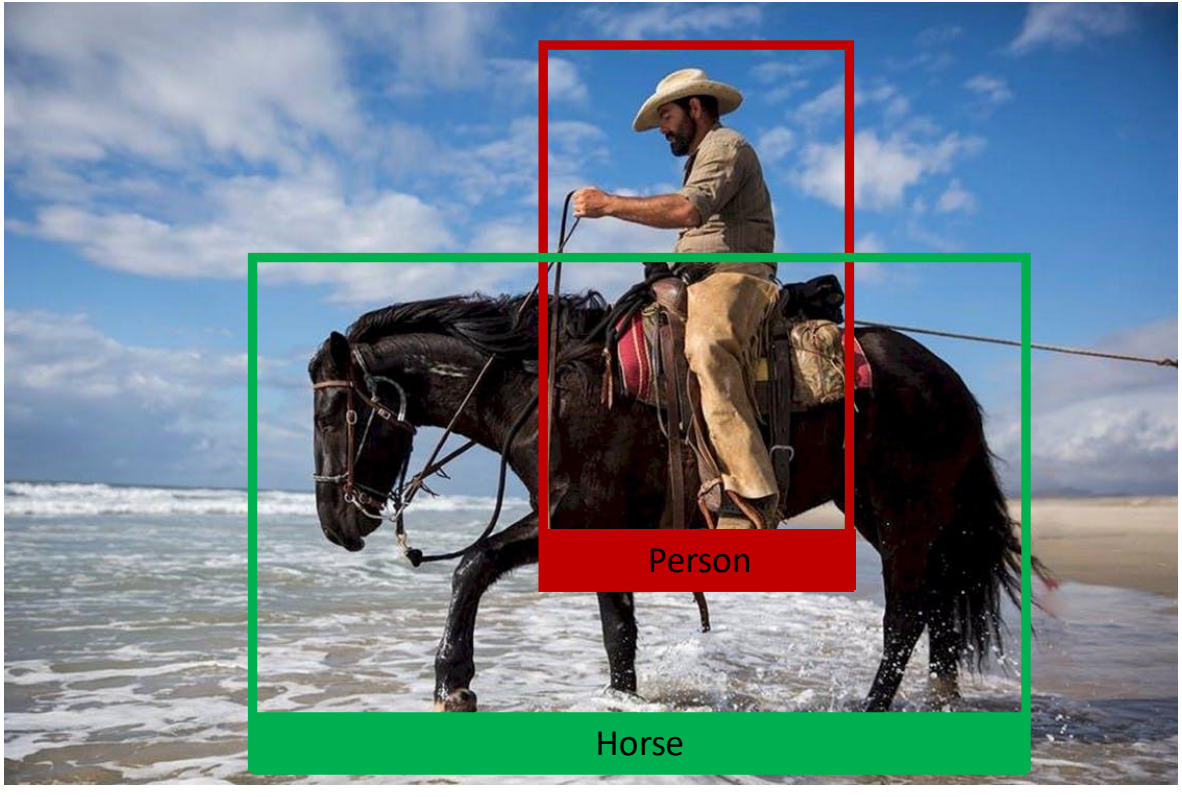
1. Viewpoint variations ← diff views
2. Intra-class variations ← diff types dogs
3. Fine-grained categories
4. Background clutter
5. Illumination changes
6. Deformations
7. Occlusion





Slide inspired from Justin Johnson. Image: https://www.kaggle.com/datasets/tongpython/cat-and-dog/data, https://www.shutterstock.com/search/dog-side-eye?image_type=illustration https://stockcake.com/i/dogs-in-landfill_1267699_367323

Image Classification: a Building Block for Other Tasks

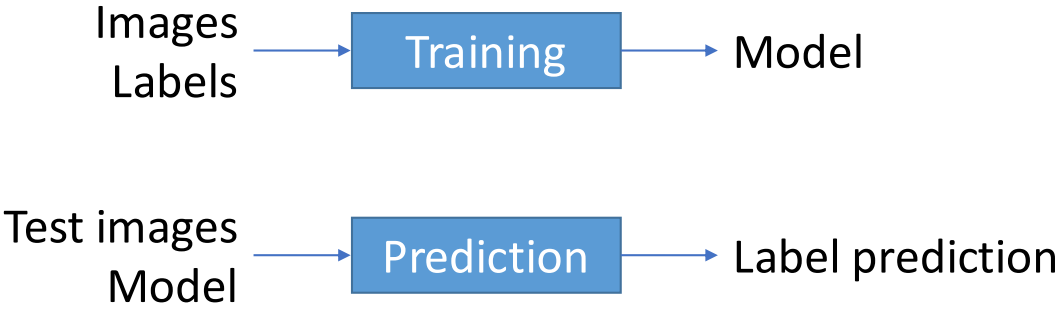


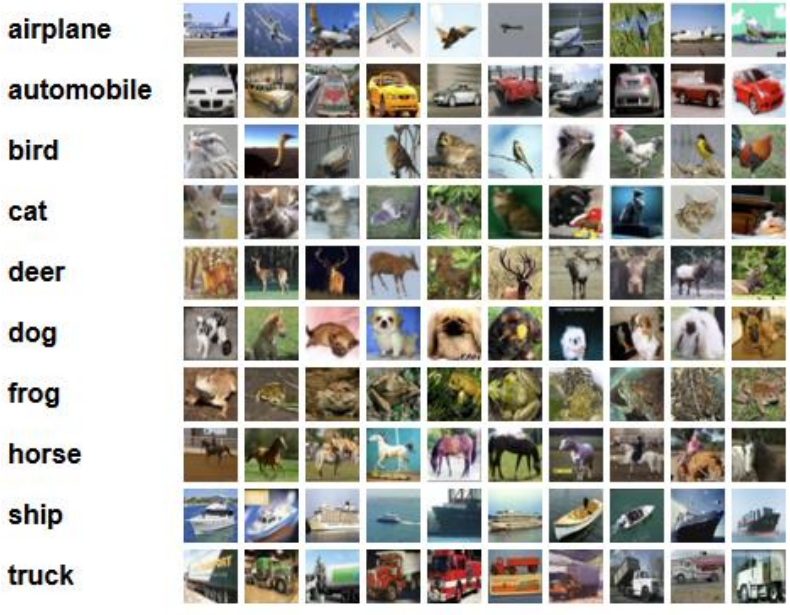
Object detection
Image captioning

Machine Learning Approach for Classification

Unlike, e.g., sorting a list of numbers, no obvious way to hard-code the algorithm for recognizing a cat, or other classes.
We must go for a machine learning (data-driven) approach.

1. Collect a dataset of images and labels
2. Use Machine Learning to train a classifier
3. Evaluate the classifier on new images





Slide inspired from Justin Johnson.

Alex Krizhevsky, "Learning Multiple Layers of Features from Tiny Images", Technical Report, 2009.

Image Classification Datasets: MNIST

10 classes: digits 0 to 9
28×28 grayscale images
50k training images
10k test images

Results from MNIST often do not hold on more complex datasets!

Slide inspired from Justin Johnson.

Image Classification Datasets: CIFAR10

10 classes
50k training images (5k per class)
10k testing images (1k per class)
32×32 RGB images

Slide inspired from Justin Johnson. Alex Krizhevsky, “Learning Multiple Layers of Features from Tiny Images”, Technical Report, 2009.

Image Classification Datasets: CIFAR100

100 classes
50k training images (500 per class)
10k testing images (100 per class)
32×32 RGB images

20 superclasses with 5 classes each:
Aquatic mammals : beaver, dolphin, otter, seal, whale
Trees : Maple, oak, palm, pine, willow

Slide inspired from Justin Johnson. Alex Krizhevsky, “Learning Multiple Layers of Features from Tiny Images”, Technical Report, 2009.

Image Classification Datasets: ImageNet

1000 classes

~1.3M training images (~1.3K per class) 50K validation images (50 per class) 100K test images (100 per class) test labels are secret!

Images have variable size, but often resized to 256x256 for training

There is also a 22k category version of ImageNet, but less commonly used

Slide inspired from Justin Johnson. Deng et al, "ImageNet: A Large-Scale Hierarchical Image Database", CVPR 2009. Russakovsky et al, "ImageNet Large Scale Visual Recognition Challenge", IJCV 2015

Image Classification Datasets: MIT Places

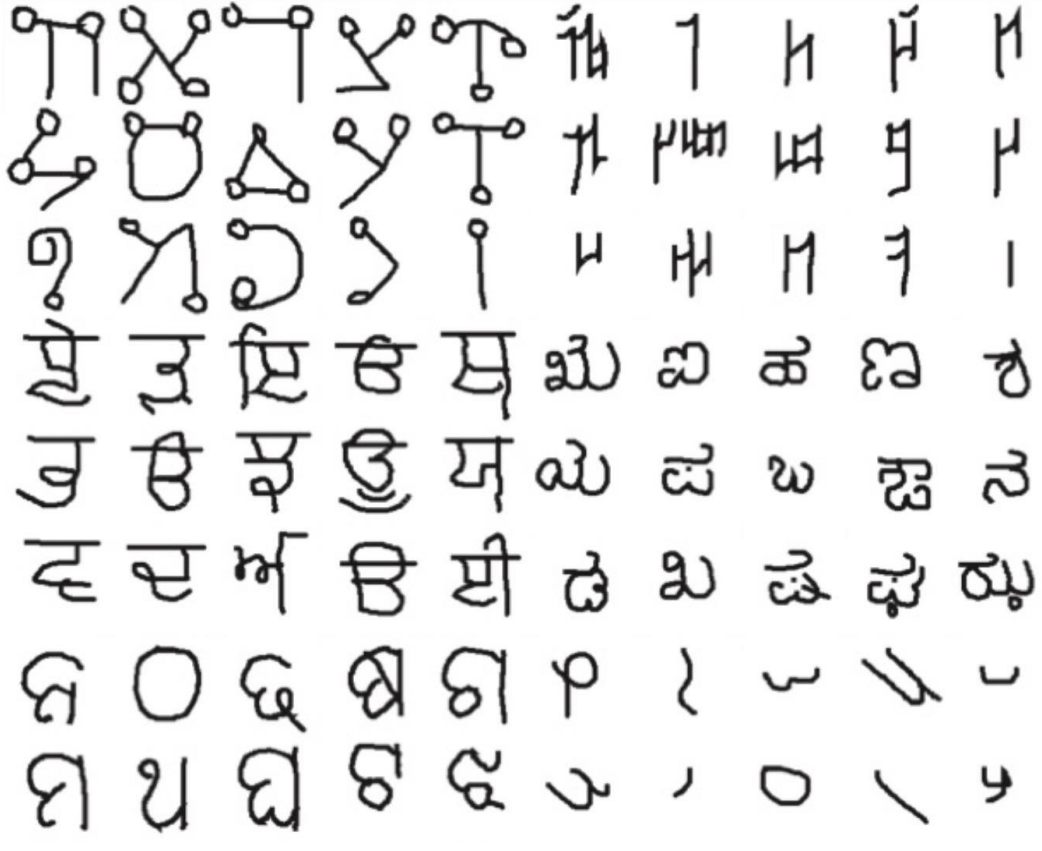
365 classes of different scene types

~8M training images
18.25K val images (50 per class)
328.5K test images (900 per class)

Images have variable size, often resize to 256x256 for training

Slide inspired from Justin Johnson. Zhou et al, “Places: A 10 million Image Database for Scene Recognition”, TPAMI 2017

Image Classification Datasets: Omniglot



1623 categories: characters from 50 different alphabets

20 images per category

Meant to test few shot learning

Slide inspired from Justin Johnson. Lake et al, “Human-level concept learning through probabilistic program induction”, Science, 2015

Nearest Neighbor Classifier

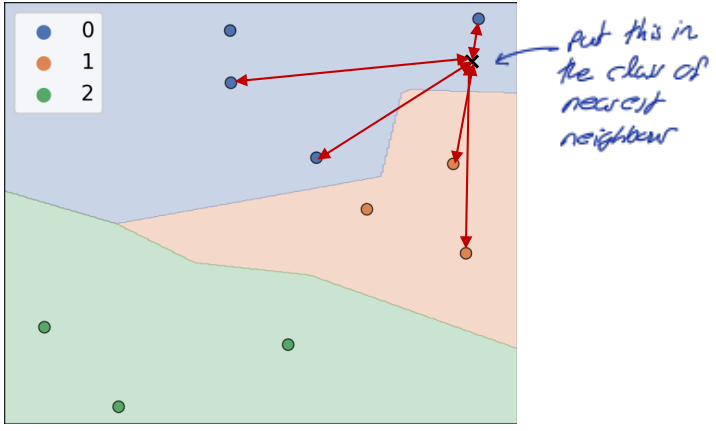
Memorizes all data and labels
Predict the label of the most similar training image

Q: With N examples, how fast is training?
A: O(1) no training

Q: With N examples, how fast is testing?
A: O(N) testing (calc dist) takes a lot of time

This is bad: We can afford slow training, but we need fast testing!

There are many methods for fast, approximate nearest neighbors:
E.g. see https://github.com/facebookresearch/faiss



Slide inspired from Justin Johnson.

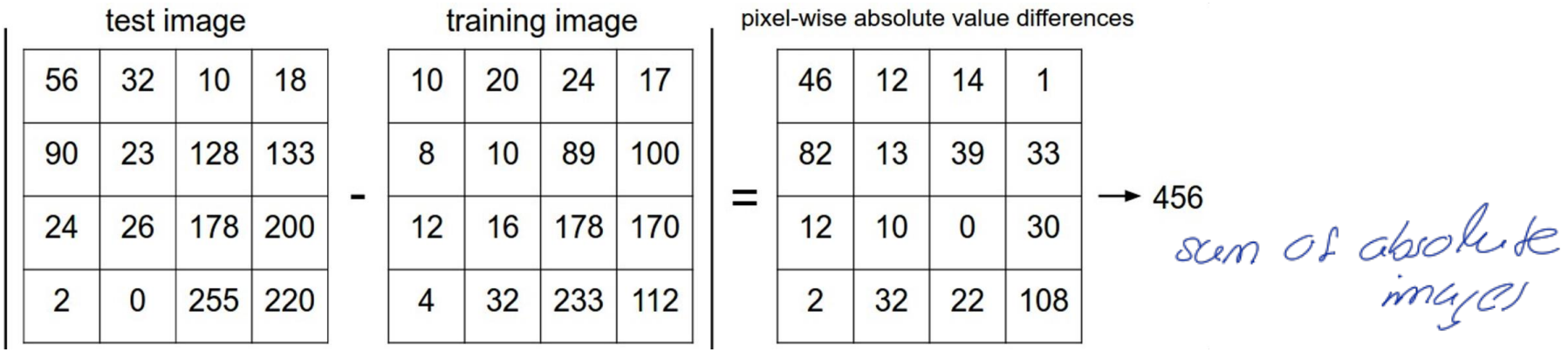
nearest neighbor is not accurate

Slide inspired from Justin Johnson.

Distance Metric to Compare Images: E.g., L1 Distance

what is dist between 2 imgs

L1 distance: $D(x, y) = \sum_{i=1}^{n} |x_i - y_i|$

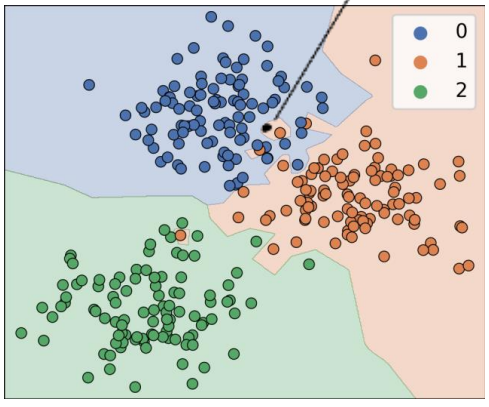


sum of absolute images

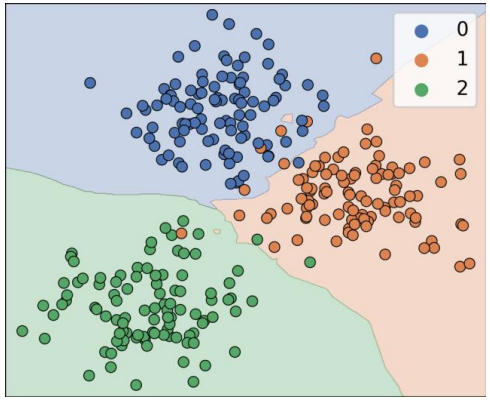
issue → if different size

Slide inspired from Justin Johnson.

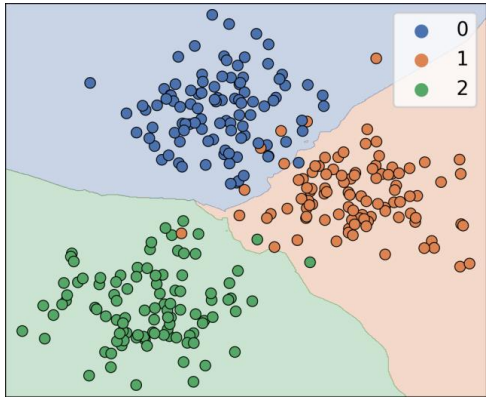
this is blue but classified as red



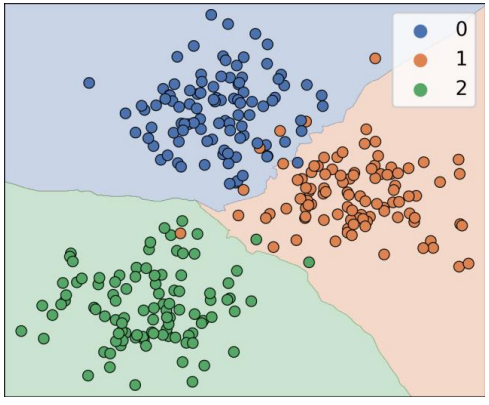
$k = 1$



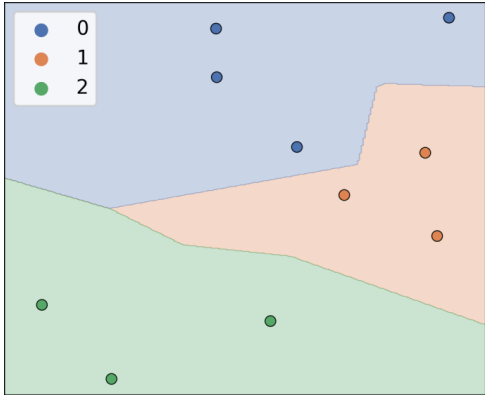
$k = 3$



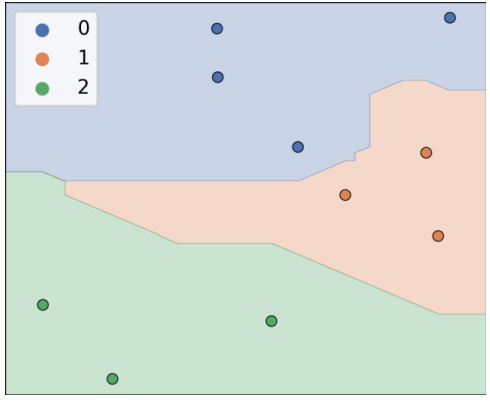
$k = 5$



$k = 7$



$k = 1, p = 2$



$k = 3, p = 1$

Minkowski Distance:

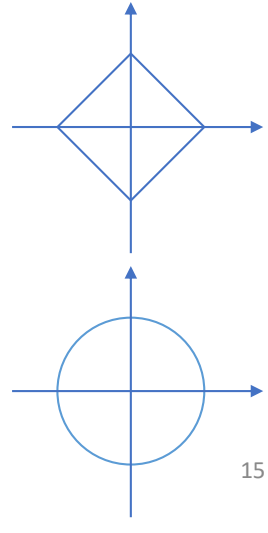
$$D(x,y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p\right)^{1/p}$$

Manhattan distance (L1, $p = 1$)

$$D(x,y) = \sum_{i=1}^{n} |x_i - y_i|$$

Euclidean Distance (L2, $p = 2$)

$$D(x,y) = \left(\sum_{i=1}^{n} |x_i - y_i|^2\right)^{1/2}$$



noisy boundaries

$k = 1$ no training error but testing error is there

boundary get smooth

$K = 7$ there is training error but less testing error.

kNN Hyperparameters

kNN Hyperparameters

What is the best value of K to use?
What is the best distance metric to use?

} set part of the learning process

These are examples of hyperparameters: choices about our learning algorithm that we don't learn from the training data; instead we set them at the start of the learning process

With the right choice of distance metric, we can apply K-Nearest Neighbor to any type of data

Choosing Hyperparameters

Slide inspired from Justin Johnson.

Idea #1: Choose hyperparameters that work best on the data

BAD: K = 1 always works perfectly on training data

Idea #2: Split data into train and test, choose hyperparameters that work best on test data

BAD: No idea how algorithm will perform on new data

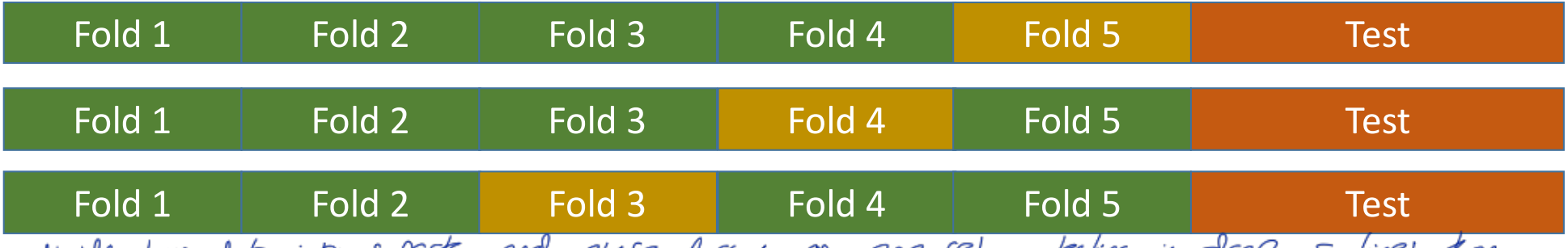
so test data must never be exposed during training

Idea #3: Split data into train, val, and test; choose hyperparameters on val and evaluate on test

Better

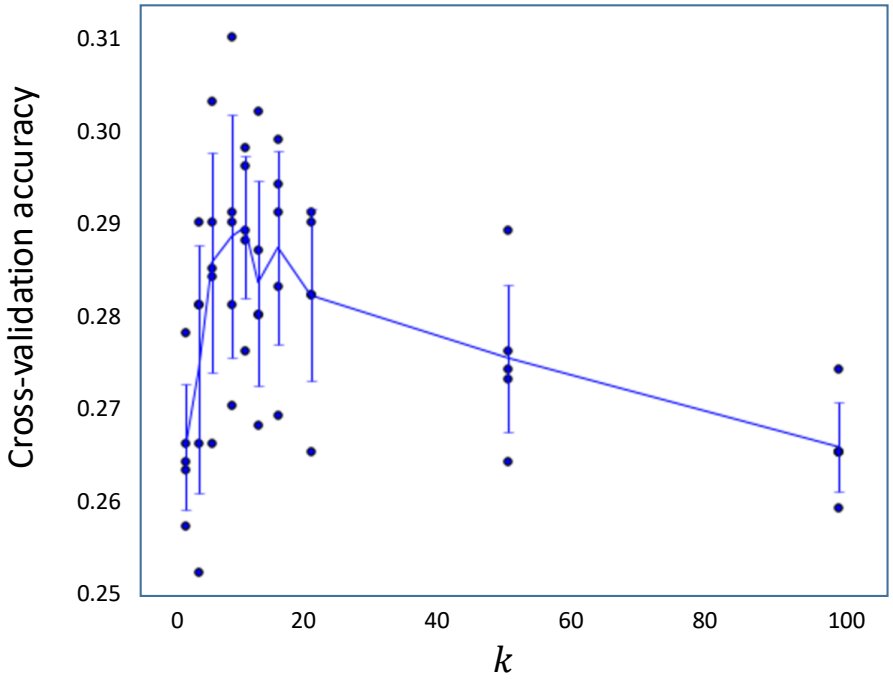
Idea #4: Cross-Validation: Split data into folds, try each fold as validation and average the results

Useful for small datasets, but (unfortunately) not used too frequently in deep learning



divide train data into 5 parts. and choose params on one fold. testing is done 5 times then.

Cross Validation to Select k



Example of 5-fold cross-validation for the value of k.

Each point: single outcome.

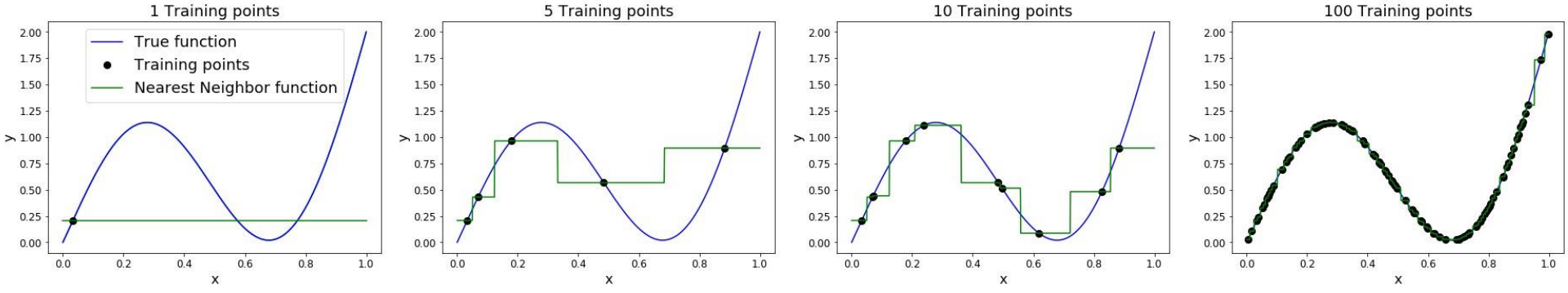
The line goes through the mean, bars indicated standard deviation

(Seems that k ~ 7 works best for this data)

Slide inspired from Justin Johnson.

K Nearest Neighbor: Universal Approximation

As the number of training samples goes to infinity, nearest neighbor can represent any(*) function!



(*) Subject to many technical conditions. Only continuous functions on a compact domain; need to make assumptions about spacing of training points; etc.

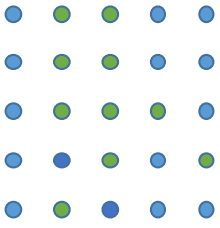
Slide inspired from Justin Johnson.

Problem: Curse of Dimensionality

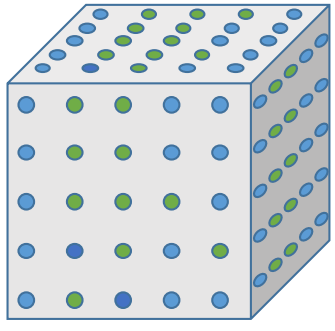
Curse of dimensionality: For uniform coverage of space, number of training points needed grows exponentially with dimension

Dimensions = 1
Points = 5

Dimensions = 2
Points = 5^2



Dimensions = 3
Points = 5^3



ing is binary
1 pixel ⇒ $2^1 = 2$
2 pixels ⇒ $2^2 = 4$

Number of possible 32x32 binary images:
$2^{32x32} \approx 10^{308}$

K-Nearest Neighbor on raw pixels is seldom used

1. Very slow at test time
2. Distance metrics on pixels are not informative

Nearest neighbor with conv. nets. work well.

Original

Boxed

Shifted

Tinted

All 3 images have same L2 distance to the one on the left

Summary

- In image classification we start with a training set of images and labels, and must predict labels on the test set.
- Image classification is challenging due to the semantic gap: we need invariance to occlusion, deformation, lighting, intraclass variation, etc.
- Image classification is a building block for other vision tasks.
- The K-Nearest Neighbors classifier predicts labels based on nearest training examples Distance metric and K are hyperparameters
- Choose hyperparameters using the validation set; only run on the test set once at the very end!
- Decision boundaries can be noisy; affected by outliers
- Instead of copying label from nearest neighbor, take majority vote from K closest points
- Using more neighbors helps smooth out rough decision boundaries.